PROYECTO FINAL BACKEND · SOLUCIONES PARA EL CLENA

Arquitectura del sistema Conserva

P3 · Monitoreo y alertas de conservación del Archivo Histórico
Modelo C4 (niveles 1 a 4) y decisiones técnicas — Entregable E5

Proyecto	P3 · Monitoreo y alertas de conservación (equipo de origen: G7)
Equipo	Equipo P3 · Backend 2026
Integrantes	Gabriel Caicedo de la Iglesia (Product Owner) Leonel José Triana Martínez (Scrum Master) Víctor Cantillo Valencia (Development Team) Sebastián González Quintanilla (Development Team)
Curso	Backend · Ingeniería de Sistemas · Universidad del Norte
Repositorio	github.com/mogeko-cpu/Proyecto-Final-Backend
Fecha	11 de noviembre de 2026
Versión	1.0

Índice

1. Propósito y alcance 3

2. Estilo arquitectónico 4

3. Nivel 1 · Contexto 5

4. Nivel 2 · Contenedores 7

5. Nivel 3 · Componentes de la API 9

6. Nivel 4 · Código (inyección de dependencias) 11

7. Estructura de carpetas 15

8. Decisiones técnicas 16

9. Seguridad, observabilidad y pruebas 18

10. Fuera de alcance 19

Índice de figuras

Figura 1. Diagrama de contexto (C4 nivel 1) 6

Figura 2. Diagrama de contenedores (C4 nivel 2) 8

Figura 3. Componentes de la API por capa (C4 nivel 3) 10

Figura 4. Inyección del repositorio en CreateRecord (C4 nivel 4) 12

1. Propósito y alcance

Este documento describe la arquitectura de **Conserva**, el sistema que reemplaza la evaluación visual, manual y esporádica del estado de los documentos del Archivo Histórico del CLENA por un registro con historial, monitoreo ambiental y alertas tempranas. Corresponde al entregable E5 de la Entrega 1.

La arquitectura se documenta con el modelo C4: contexto (nivel 1), contenedores (nivel 2), componentes de la API (nivel 3) y, de forma opcional, el código de un caso de uso (nivel 4). Todos los diagramas se elaboraron en draw.io; sus fuentes editables están en `docs/diagramas/` y aquí se incrustan como imagen.

El alcance funcional cubierto por la arquitectura es:

- Inventario de expedientes con ubicación física (depósito, estantería y caja) y depósitos con rangos ambientales configurables.
- Inspecciones con estado Óptimo, Regular o Crítico, hallazgos (humedad, plagas, desgaste) e historial inmutable por expediente.
- Motor de reglas que genera alertas por estado crítico, inspección vencida y lectura ambiental fuera de rango, sin duplicarlas.
- Ingesta de lecturas de temperatura y humedad por API y un simulador de sensores.
- Dashboard con alertas priorizadas y asignación y seguimiento de tareas de intervención.
- Requisitos transversales: autenticación JWT con refresh, tres roles (ADMIN, STAFF, PUBLIC), aviso de privacidad con consentimiento y eliminación de cuenta.

2. Estilo arquitectónico

El backend sigue **Clean Architecture** en cuatro capas. La regla central es que las dependencias del código fuente apuntan siempre hacia el dominio:

Capa	Carpeta	Responsabilidad	Puede depender de
Dominio	`backend/src/domain`	Entidades, reglas de negocio puras y puertos (interfaces `Repository`, `Runtime` y `Security`).	Nada
Aplicación	`backend/src/application`	Casos de uso: orquestan reglas y puertos. Son clases TypeScript sin decoradores.	Dominio
Infraestructura	`backend/src/infrastructure`, `backend/src/modules`	Adaptadores concretos (PostgreSQL con TypeORM, JWT y bcrypt, reloj del sistema), migraciones, datos semilla y la raíz de composición de NestJS.	Dominio, Aplicación
Presentación	`backend/src/presentation`	Controladores HTTP, DTOs con validación, guard de acceso y filtro de errores.	Aplicación, Dominio

> **Cómo se verifica.** Ningún archivo de `domain/` ni de `application/` importa `@nestjs/*`, `typeorm` ni `express`. Se comprueba con `grep -rn "from '@nestjs\|from 'typeorm\|from 'express" backend/src/domain backend/src/application`, que no devuelve resultados. Gracias a esto, las pruebas unitarias ejecutan los casos de uso con un repositorio en memoria (`backend/test/memory.ts`), sin base de datos ni NestJS.

El código se organiza además por **módulo funcional**: autenticación, inventario, monitoreo e intervenciones. Cada módulo tiene un archivo en cada capa (por ejemplo, `application/inventory.ts`, `presentation/controllers/inventory.controller.ts` y `modules/inventory.module.ts`) y se registra con una línea en `app.module.ts`. Así, cada integrante puede trabajar y explicar su módulo completo sin tocar los de los demás.

3. Nivel 1 · Contexto

El nivel de contexto (Figura 1) muestra a Conserva como una sola caja, los tres actores definidos por el curso y el sistema externo que alimenta el monitoreo ambiental.

Elemento	Tipo	Relación con Conserva
Administrador	Persona	Configura depósitos y sus rangos de temperatura y humedad, crea cuentas institucionales y archiva expedientes. Usa la aplicación web por HTTP/HTTPS.
Personal del CLENA	Persona	Archivistas, restauradores y coordinación: registran expedientes e inspecciones, atienden alertas y ejecutan tareas de intervención.
Usuario público	Persona	Crea su cuenta aceptando el aviso de privacidad, consulta su perfil y puede eliminar sus datos. No accede al inventario.
Simulador de sensores	Sistema externo	Script Node.js que se autentica y envía lecturas de temperatura y humedad a la API (HTTP/JSON con JWT), como lo haría un sensor IoT real.

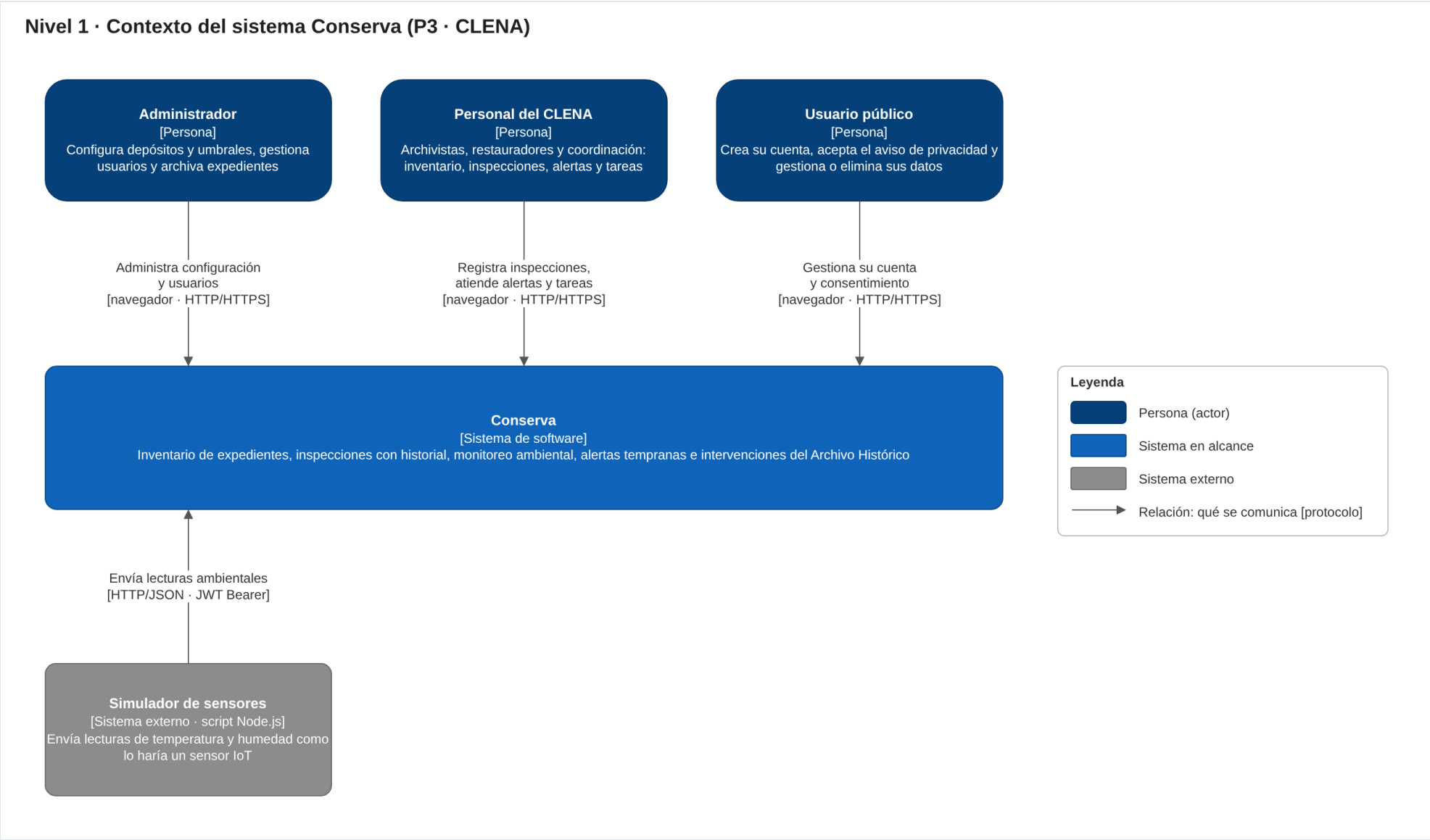


Figura 1. Diagrama de contexto del sistema Conserva (C4 nivel 1)

4. Nivel 2 · Contenedores

La Figura 2 muestra los contenedores. El sistema se despliega con `docker compose` en cuatro contenedores. El navegador solo habla con Nginx, que sirve la SPA y reenvía `/api`, `/health` y `/docs` a la API.

Contenedor	Tecnología	Responsabilidad	Puerto
Aplicación web	React 19, React Router 7, Vite 7; servida por Nginx 1.28	Interfaz de usuario. Nginx aplica cabeceras de seguridad (CSP, X-Frame-Options) y actúa como proxy inverso hacia la API.	8080 → 80
API REST	NestJS 11 sobre Node.js 24	Casos de uso, reglas, autenticación y autorización, documentación OpenAPI en `/docs` y evaluación periódica de inspecciones vencidas.	3000 (solo localhost)
Base de datos	PostgreSQL 17	Persistencia relacional con claves foráneas, restricciones únicas y de chequeo e índices de búsqueda.	5432 (red interna)
Inicializador de secretos	Node.js 24 (`scripts/init-secrets.cjs`)	Se ejecuta una vez: genera la contraseña de la base de datos y las claves JWT en el volumen `config`. Así no hay secretos en el repositorio.	—

Protocolos. Navegador → Nginx: HTTP (HTTPS en un despliegue público). Nginx → API: HTTP/JSON con el token JWT en la cabecera `Authorization: Bearer`. API → PostgreSQL: SQL mediante TypeORM y el driver `pg` sobre TCP 5432. Simulador → API: HTTP/JSON con JWT. Los secretos se comparten como archivos de solo lectura en un volumen Docker.

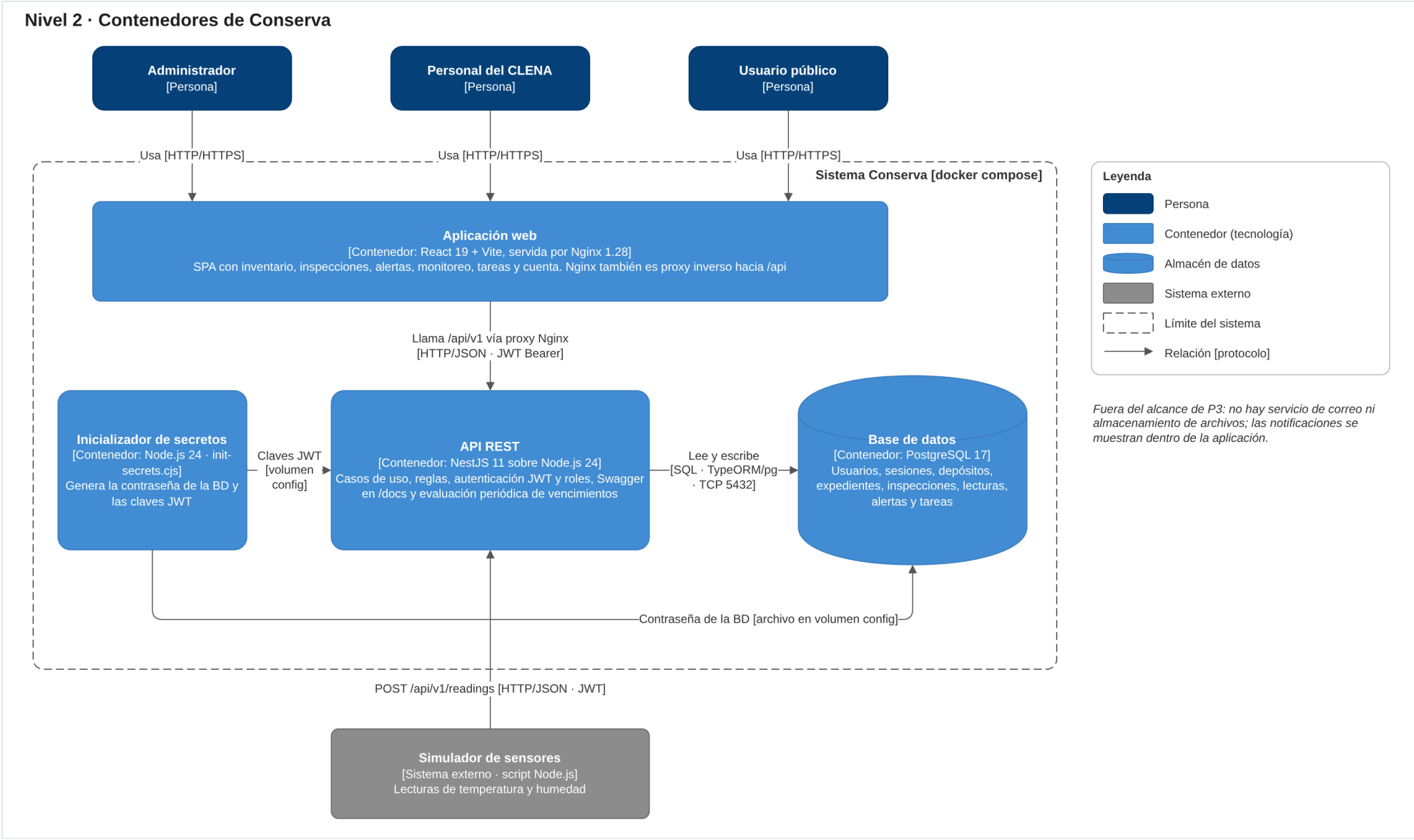


Figura 2. Diagrama de contenedores de Conserva (C4 nivel 2)

5. Nivel 3 · Componentes de la API

El nivel 3 (Figura 3) descompone la API en componentes agrupados por capa, de izquierda a derecha: Presentación, Aplicación, Dominio e Infraestructura. Las flechas sólidas son dependencias o llamadas; las punteadas con punta hueca indican que un adaptador de infraestructura implementa una interfaz del dominio. Ninguna flecha sale del dominio: tanto la presentación y la aplicación (desde la izquierda) como la infraestructura (desde la derecha) apuntan hacia él.

Flujo de una petición

1. El **AccessGuard** global valida el JWT con el caso de uso `Authenticate` y comprueba el rol exigido por `@Roles()`. Las rutas marcadas con `@Public()` (login, registro, refresh y salud) no requieren sesión.
2. El **ValidationPipe** convierte el cuerpo en un DTO y lo valida con class-validator; rechaza campos no declarados.
3. El **controlador** llama a un caso de uso. Por ejemplo, `POST /api/v1/records/:id/inspections` invoca `RegisterInspection` con el ID del usuario de la sesión como inspector.
4. El **caso de uso** aplica las reglas del dominio y usa el puerto `Repository`; las operaciones compuestas (guardar la inspección, actualizar el expediente y abrir o resolver la alerta) se ejecutan dentro de `repo.transaction()`.
5. Si algo falla, el **ErrorFilter** responde siempre con `{statusCode, message, path, timestamp}` y traduce los errores de PostgreSQL (código duplicado → 409, referencia inexistente → 400).

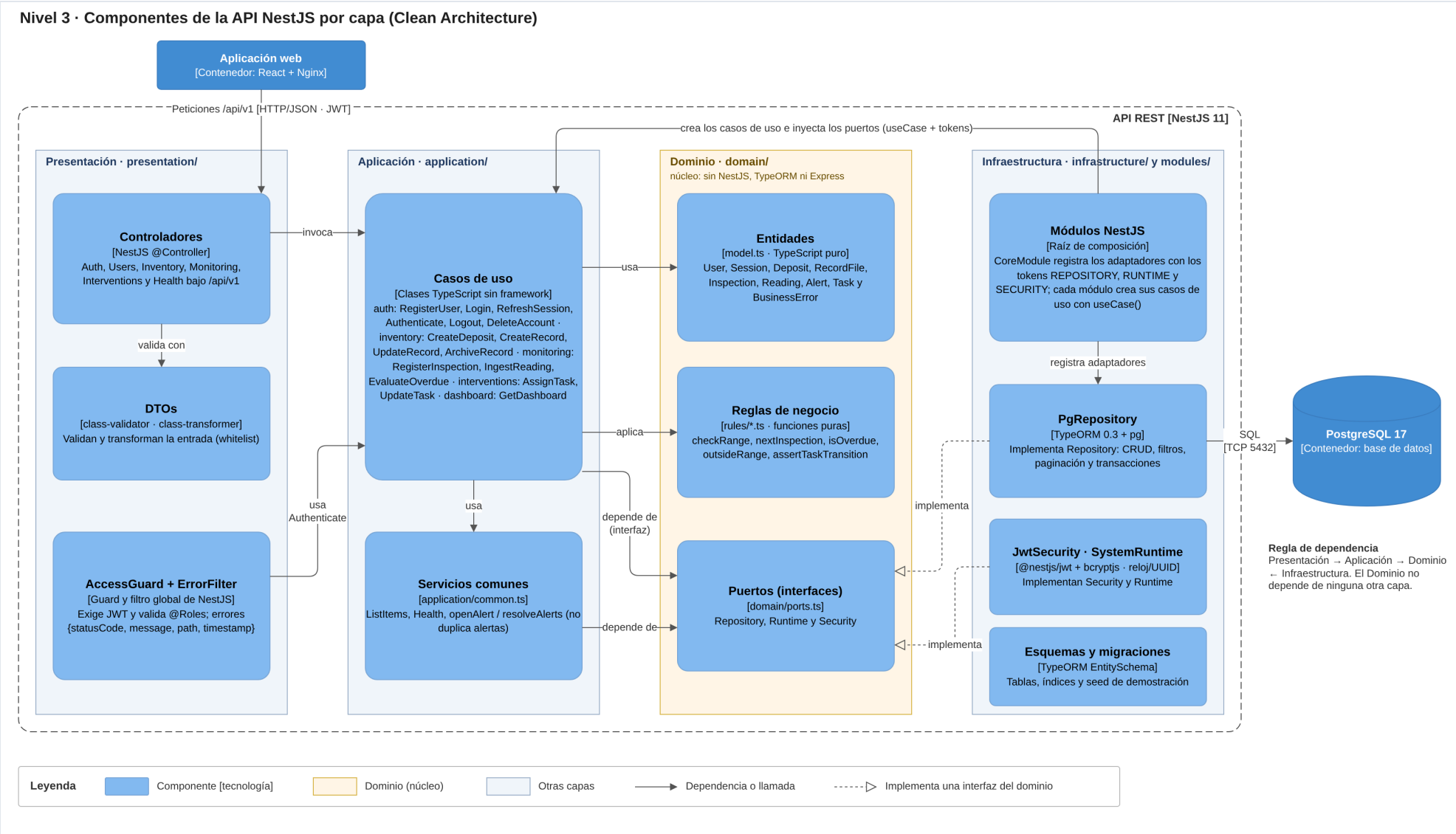


Figura 3. Componentes de la API NestJS agrupados en las cuatro capas de Clean Architecture (C4 nivel 3)

6. Nivel 4 · Código (inyección de dependencias)

El nivel 4 (Figura 4) muestra cómo un caso de uso recibe su repositorio sin conocer la implementación. `CreateRecord` declara en su constructor la interfaz `Repository`; NestJS lo crea mediante la función `useCase()`, que le pasa los objetos registrados con los tokens `REPOSITORY` y `RUNTIME`.

Nivel 4 · Código: cómo CreateRecord recibe su repositorio por token de inyección

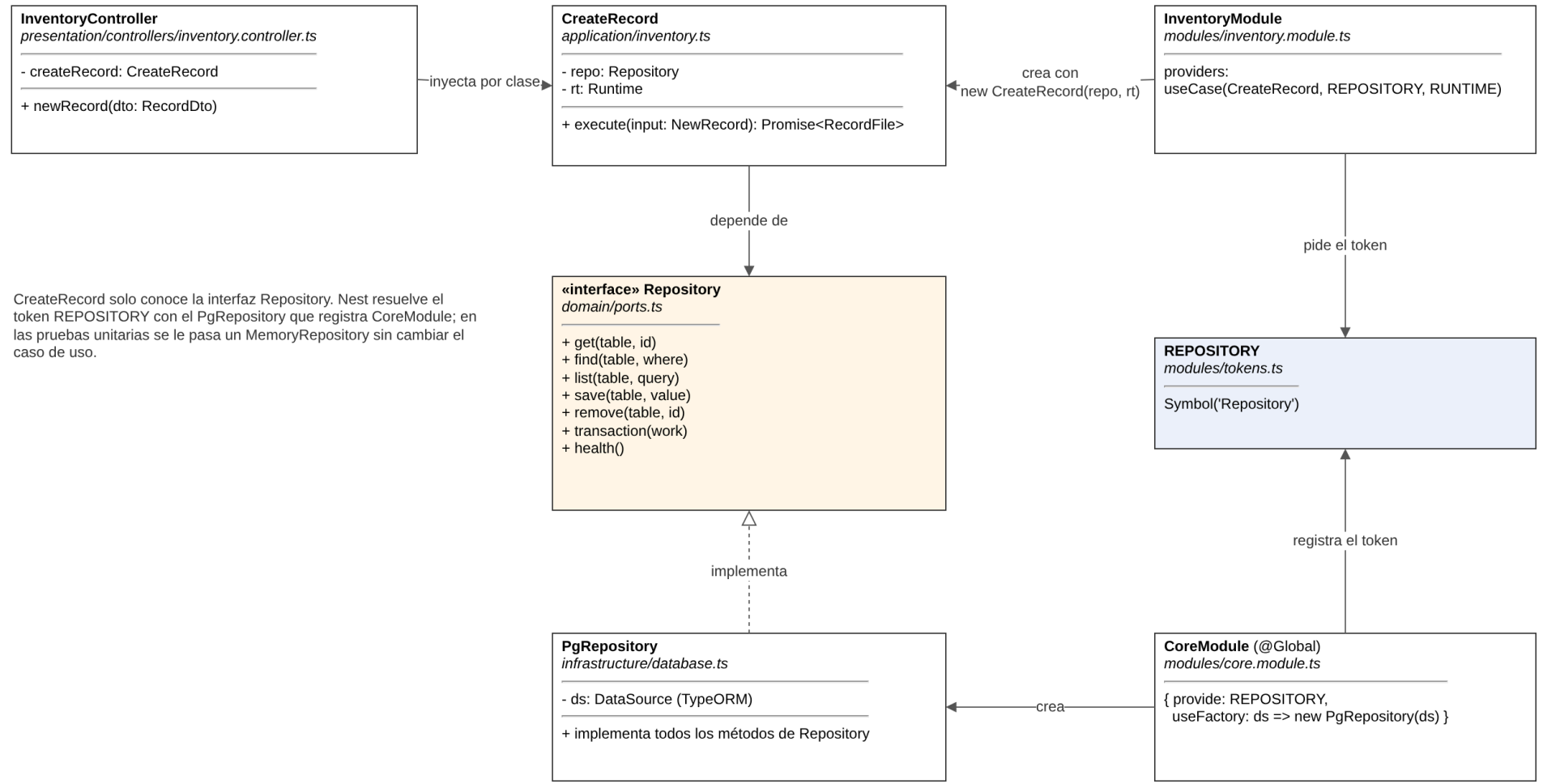


Figura 4. Inyección del repositorio en el caso de uso CreateRecord (C4 nivel 4)

Caso de uso: depende solo del puerto

```
export class CreateRecord {
  constructor(
    private repo: Repository,
    private rt: Runtime,
  ) {}
  async execute(input: NewRecord) {
    requireFound(await this.repo.get('deposits', input.depositId), 'Depósito');
    return this.repo.save('records', {
      ...input,
      id: this.rt.id(),
      createdAt: this.rt.now(),
      condition: 'REGULAR',
      archived: false,
      // Queda pendiente de primera inspección desde el momento de su registro.
      nextInspectionAt: this.rt.now(),
    });
  }
}
```

Raíz de composición: tokens y adaptador

```
/** Tokens de inyección de los puertos del dominio (ver domain/ports.ts). */
export const REPOSITORY = Symbol('Repository');
export const RUNTIME = Symbol('Runtime');
export const SECURITY = Symbol('Security');
```

```

/** Une los puertos del dominio con sus adaptadores (PostgreSQL, reloj del sistema y JWT). */
@Global()
@Module({
  providers: [
    {
      provide: DataSource,
      useFactory: async () => {
        const ds = createDataSource();
        await ds.initialize();
        await ds.runMigrations();
        return ds;
      },
    },
    {
      provide: REPOSITORY,
      useFactory: (ds: DataSource) => new PgRepository(ds),
      inject: [DataSource],
    },
    { provide: RUNTIME, useClass: SystemRuntime },
```

Módulo que une el caso de uso con el repositorio

```
import { FactoryProvider } from '@nestjs/common';

type UseCaseClass<T> = new (...args: never[]) => T;

/**
 * Registra un caso de uso en Nest sin que la capa de aplicación dependa del framework:
 * Nest crea la clase pasando los puertos indicados (repositorio, reloj, seguridad).
 */
export function useCase<T>(type: UseCaseClass<T>, ...inject: symbol[]): FactoryProvider<T> {
  return {
    provide: type,
    useFactory: (...deps: unknown[]) => new type(...(deps as never[])),
    inject,
  };
}
```

```
import { Module } from '@nestjs/common';
import {
  ArchiveRecord,
  CreateDeposit,
  CreateRecord,
  GetRecord,
  UpdateDeposit,
  UpdateRecord,
} from '../application/inventory';
import { InventoryController } from '../presentation/controllers/inventory.controller';
import { REPOSITORY, RUNTIME } from './tokens';
import { useCase } from './use-case';

@Module({
  controllers: [InventoryController],
  providers: [
    useCase(CreateDeposit, REPOSITORY, RUNTIME),
    useCase(UpdateDeposit, REPOSITORY),
    useCase(CreateRecord, REPOSITORY, RUNTIME),
    useCase(UpdateRecord, REPOSITORY, RUNTIME),
    useCase(ArchiveRecord, REPOSITORY, RUNTIME),
    useCase(GetRecord, REPOSITORY),
  ],
})
export class InventoryModule {}
```

7. Estructura de carpetas

Estructura real del repositorio (monorepo con workspaces de npm). Se omiten `node_modules` y los archivos de compilación.

```
Proyecto-Final-Backend/
├── .github/                              # CI y plantillas de PR e issue
├── backend/                              # API NestJS
│   ├── src/
│   │   ├── application/                  # casos de uso por módulo
│   │   │   ├── auth.ts
│   │   │   ├── common.ts
│   │   │   ├── dashboard.ts
│   │   │   ├── interventions.ts
│   │   │   ├── inventory.ts
│   │   │   └── monitoring.ts
│   │   ├── domain/                       # núcleo: sin imports de framework
│   │   │   ├── rules/                    # reglas puras por módulo
│   │   │   ├── model.ts                  # entidades y BusinessError
│   │   │   └── ports.ts                  # interfaces Repository, Runtime, Security
│   │   ├── infrastructure/               # adaptadores
│   │   │   ├── migrations/               # migraciones versionadas
│   │   │   ├── database.ts               # PgRepository (TypeORM) y DataSource
│   │   │   ├── schema.ts                 # EntitySchema de cada tabla
│   │   │   ├── security.ts               # JwtSecurity (JWT + bcrypt) y SystemRuntime
│   │   │   └── seed.ts                   # datos ficticios de demostración
│   │   ├── modules/                      # raíz de composición de NestJS
│   │   │   ├── auth.module.ts
│   │   │   ├── core.module.ts
│   │   │   ├── interventions.module.ts
│   │   │   ├── inventory.module.ts
│   │   │   ├── monitoring.module.ts
│   │   │   ├── tokens.ts
│   │   │   └── use-case.ts
│   │   ├── presentation/                 # HTTP
│   │   │   ├── controllers/              # un controlador por módulo
│   │   │   ├── dto/                      # DTOs con class-validator
│   │   │   ├── access.guard.ts           # guard global: JWT y roles
│   │   │   ├── http.ts                   # ErrorFilter, @Public, @Roles
│   │   │   └── openapi.ts                # respuestas comunes en Swagger
│   │   ├── app.module.ts                 # CoreModule + módulos funcionales
│   │   ├── bootstrap.ts                  # helmet, CORS, validación, errores, Swagger
│   │   └── main.ts                       # arranque: logger JSON y puerto
│   ├── test/                             # unitarias (*.spec.ts) y e2e/
│   ├── package.json
│   ├── tsconfig.build.json
│   └── tsconfig.json
├── docs/                                 # informes en PDF (E3, E4, E5, Scrum)
│   ├── diagramas/                        # fuentes .drawio de los C4
│   └── arquitectura.pdf
├── frontend/                             # SPA React + Vite
│   ├── src/
│   │   ├── pages/                        # una página por pantalla
│   │   ├── App.tsx
│   │   ├── Layout.tsx
│   │   ├── ScrollToTop.tsx
│   │   ├── api.ts                        # cliente HTTP y renovación de token
│   │   ├── main.tsx
│   │   ├── navigation.ts                 # menú lateral
│   │   ├── session.tsx
│   │   ├── styles.css
│   │   └── ui.tsx                        # componentes reutilizables
│   ├── index.html
│   ├── nginx.conf
│   ├── package.json
│   ├── tsconfig.json
│   └── vite.config.ts
├── scripts/                              # secretos, admin, simulador, e2e Docker
├── .env.example
├── Dockerfile
├── README.md
├── compose.yaml                          # init, db, backend y frontend
├── eslint.config.mjs
└── package.json
```

8. Decisiones técnicas

Decisión	Alternativa considerada	Justificación
NestJS 11 como framework de la API	Express sin framework	Exigido por el curso. Aporta módulos, inyección de dependencias, guards, pipes y generación de OpenAPI, que se usan solo en las capas externas.
Casos de uso como **clases sin decoradores** registradas con `useCase()`	Servicios con `@Injectable()`	Mantiene la capa de aplicación independiente del framework y permite probarla con dobles en memoria. El costo es una línea de registro por caso de uso en su módulo.
TypeORM 0.3 con EntitySchema	Entidades con decoradores de TypeORM; Prisma	Con `EntitySchema` el mapeo vive en `infrastructure/schema.ts` y las entidades del dominio son interfaces TypeScript puras, sin imports del ORM. Integración nativa con NestJS y migraciones en TypeScript.
Migraciones versionadas y `synchronize: false`	Sincronización automática del esquema	El esquema de producción solo cambia mediante migraciones revisadas en un PR. La API las aplica al iniciar.
PostgreSQL 17	MySQL; MongoDB	Integridad referencial, restricciones únicas y de chequeo (por ejemplo, humedad entre 0 y 100 %), transacciones para inspección más alerta, y `timestamptz` para lecturas y vencimientos.
Puerto `Repository` genérico por tabla	Un repositorio por agregado	Un solo adaptador implementa CRUD, filtros, búsqueda, ordenamiento validado y paginación para todas las tablas, y comparte la misma transacción entre ellas. El dominio sigue sin conocer TypeORM.
class-validator y class-transformer	Validación manual; Zod	Integración directa con el `ValidationPipe` de NestJS y con Swagger. Se usa `whitelist` y `forbidNonWhitelisted` para rechazar campos inesperados (por ejemplo, un `role` en el registro público).
JWT de acceso (15 min) y refresh (7 días) con rotación	Sesiones con cookie	API sin estado para la SPA y el simulador. Cada refresh crea una sesión nueva y revoca la anterior; en la tabla `sessions` se guarda un digest del token, no el token. El logout y la eliminación de cuenta revocan las sesiones.
bcryptjs (costo 12)	bcrypt nativo; argon2	Implementación en JavaScript puro: compila sin dependencias nativas en la imagen Alpine y en Windows.

Logger de NestJS en formato JSON con `requestId`	Winston; Pino	`ConsoleLogger({ json: true })` no agrega dependencias. Cada petición registra método, ruta, estado y duración, y devuelve `X-Request-Id` para rastrear errores.
Evaluación de vencidas con **temporizador del módulo** (60 s) y endpoint manual	`@nestjs/schedule`	El temporizador vive en `MonitoringModule` (infraestructura) y solo invoca el caso de uso `EvaluateOverdue`, que es idempotente. `POST /api/v1/alerts/evaluate` permite ejecutarlo en la demo.
React 19 + Vite 7, sin librería de componentes; íconos **lucide-react**	Material UI; Bootstrap	Control total sobre accesibilidad (etiquetas, foco, contraste) y un bundle liviano. Los componentes reutilizables están en `frontend/src/ui.tsx`.
Docker Compose con contenedor de inicialización de secretos	Variables de entorno en un `.env` versionado	`docker compose up` levanta todo en una máquina limpia sin que existan secretos en el repositorio; `.env.example` documenta las variables.
Jest + Supertest y **GitHub Actions**	Pruebas manuales	Pruebas unitarias de dominio y aplicación con cobertura mínima del 70 %, y pruebas e2e contra PostgreSQL real. El CI corre lint, pruebas y build en cada PR; `main` y `develop` exigen el check en verde.

9. Seguridad, observabilidad y pruebas

Seguridad

- Contraseñas con bcrypt; los listados de usuarios nunca devuelven `passwordHash` ni digests de tokens.
- Autorización por rol en cada ruta (guard global) y verificación de propiedad en las tareas: solo el responsable o un ADMIN puede actualizarlas.
- Cabeceras de seguridad con `helmet` en la API y CSP en Nginx; CORS limitado a `FRONTEND_ORIGIN`; límite de 120 peticiones por minuto con `@nestjs/throttler`.
- Datos personales mínimos (nombre, correo y fecha de consentimiento). La eliminación de cuenta anonimiza los datos y conserva la trazabilidad institucional; no se permite eliminar al último administrador.

Observabilidad

- `GET /api/v1/health` verifica la conexión con PostgreSQL (200 si está disponible, 503 si no). Docker usa `/health` como healthcheck.
- Logs estructurados en JSON por petición y por cada evaluación de vencidas.

Pruebas

- Unitarias: reglas de dominio y casos de uso de cada módulo con `MemoryRepository`, posible gracias a los puertos.
- E2E: cada módulo levanta la API completa contra una base cuyo nombre contiene `test`, para impedir que se ejecuten sobre datos reales.

10. Fuera de alcance

P3 no contempla, en esta entrega, un **servicio de correo** ni **almacenamiento de archivos**: las alertas y notificaciones se muestran dentro de la aplicación (dashboard y página de alertas) y no se adjuntan fotografías a las inspecciones. Por eso no aparecen en el diagrama de contenedores.

Si el alcance se amplía para incluir notificaciones por correo, la arquitectura lo admite sin cambiar el dominio: se agregaría un puerto `Notifier` en `domain/ports.ts`, un adaptador SMTP en `infrastructure/` y un contenedor de correo en el nivel 2.

Los sensores reales quedan representados por el simulador: un dispositivo IoT solo necesitaría autenticarse y llamar al mismo endpoint `POST /api/v1/readings`.